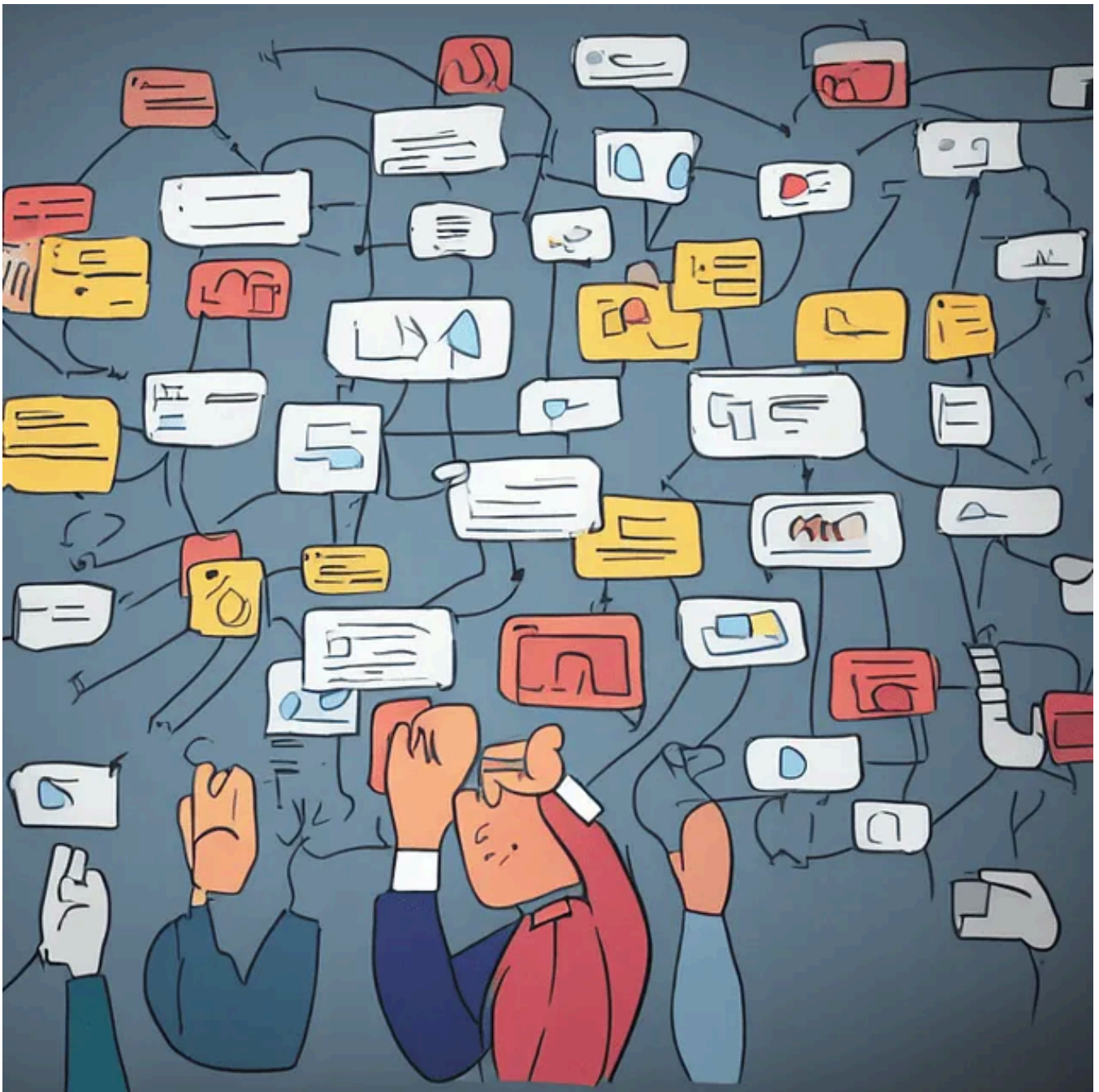




ในตอนที่แล้วเราได้รู้ว่า Normalization คืออะไรไปแล้ว ตอนนี้เรามาดูว่า Normalization มีขั้นตอนอะไรบ้าง จากนั้นเราไปรู้จัก 1NF , 2NF , 3NF, BCNF ว่ามันคืออะไร

Normalization Process

กระบวนการทำ Normalization มีขั้นตอนง่ายๆดังต่อไปนี้

1. ตรวจสอบว่า Table ที่ทำการตรวจสอบนั้นอยู่ใน 1NF ไหม ถ้าไม่อยู่ในรูปแบบ 1NF ต้องทำให้อยู่ในรูปแบบ 1NF
2. ทำการตรวจสอบว่า Table นั้นอยู่ใน NF ขั้นที่ต้องการแล้วรึยัง ถ้าไม่อยู่ให้ทำการแยก Table ตามหลักการของแต่ละ NF โดยทั่วไปแล้วเราจะทำจนกว่าจะถึงขั้น 5NF (ใครบอกถึง 3NF ถือว่าผิดนะครับ ต้องทำถึง 5NF)

3. กลับทำข้อ 2 ไปเรื่อยๆจนกว่าทุก Table ที่เราจะมีอยู่ใน NF ที่เราต้องการ (โดยทั่วไปคือต้องถึง 5NF)

อ่านถึงตรงนี้แล้วอาจจะงงเพราะศัพท์พวก NF คืออะไร 1NF คืออะไร 5 NF คืออะไร เดียวเรามาค่อยๆดูกันครับว่า NF คืออะไร ไล่ไปเรื่อยๆจนถึง BCNF แล้วไปต่อ 4NF , 5NF ในตอนต่อไป

Normal form (NF)

Normal form คือ Table (Relation) ที่อยู่ในรูปแบบที่ดี (Good form) โดยแต่ละ Normal form นั้นจะแก้ปัญหาเกี่ยวกับ INSERT UPDATE DELETE ในกรณีต่างๆ ซึ่งแต่ละ Normal form นั้นระบุเงื่อนไขไว้ว่าต้องมีคุณสมบัติอะไรจึงจะผ่าน Normal form นั้น ในกรณีไม่ผ่านก็จะมีวิธีแก้ไขบอกกว่าต้องทำอะไร

1NF

A relation table structure T is in the first normal form (1NF) , if and only if all of its attributes contain atomic values only , and each attribute belongs to only one domain

ผมแนะนำให้อ่านนิยามด้านบนครับเพราะเป็นนิยามที่ถูกต้อง แต่หากท่านไม่เข้าใจลองมาอ่านที่ผมแปลไทยดูครับ โดยคำแปลก็คือ

Table ที่จะเป็น 1NF ได้นั้น Column (Attribute) ทั้งหมดของ Table นั้นจะต้องมีค่าเป็น atomic value และ Column เหล่านั้นต้องมีค่ามาจาก Domain เดียวกัน

อ่านแบบนี้คงจะไม่เข้าใจเรามาลองดูตัวอย่าง Table ที่ไม่ผ่าน 1NF กันดีกว่า

All of its attributes contain atomic values only : Column (Attribute) ทั้งหมดของ Table นั้นจะต้องมีค่าเป็น atomic value

เรามาดูตัวอย่าง Table ที่ Column (Attribute) มีค่าไม่เป็น atomic value

USERNAME	EMAIL	MOBILE_NO
surapong	surapong007@hotmail.com	0846568789
		0815678787
		0834587890
wasinee	very_cute_girl@hotmail.com	0851231234
		0878997890

Column : MOBILE_NO ไม่เป็น Atomic value

จากตัวอย่าง Table ในภาพจะเห็นว่า Column “MOBILE_NO” นั้นไม่เป็น Atomic value เพราะมีค่าเป็น List ของเบอร์มือถือที่ User คนนั้นมี ถามว่าปัญหาที่จะเกิดขึ้นจาก Table ที่เก็บค่าแบบนี้คือ เวลาคุณอยากเพิ่มเบอร์โทรขึ้นมาคุณจะส่งยังไง มันจะไม่ใช่การสั่งแบบ INSERT เบอร์โทรศัพท์ใหม่เข้าไป แต่ต้องไปดูก่อนว่ามี Row ของ User นี้อยู่ใน Database ไหม ถ้าไม่มีจะ INSERT แต่ถ้ามีต้องเป็น UPDATE ค่าที่ Row เก่า ซึ่งคุณจะต้องทำการ Substring ตัดต่อ String เบอร์โทรศัพท์ เช่นกันถ้าต้องการลบข้อมูลโทรศัพท์คุณจะได้ไม่ได้สั่ง DELETE ROW แต่จะกลายเป็นต้องเข้าไปหา ROW ของ USER นั้นจากนั้นดึงค่าจากนั้นต้อง Replace string แล้วเอาค่าใหม่ใส่เข้าไปแทน

อีกทั้งถ้าคุณจำได้ Relation คือ Subset of the Cartesian product of domains ผลลัพธ์ของ Cartesian product of domains ที่ออกมาใน attribute ต่างๆจะไม่มีทางเป็น Set หรือ List ดังนั้น Table ที่เราเห็นนี้ไม่ใช่ Relation ด้วยซ้ำไป

วิธีแก้ไข

วิธีแก้ไข Table นี้ให้อยู่ในรูปแบบ 1NF ก็คือทำให้มันเป็น Atomic value คือให้ 1 row มี 1 ค่าไป ซึ่งจะได้ผลลัพธ์ออกมาดังภาพด้านล่าง

USERNAME	EMAIL	MOBILE_NO
surapong	surapong007@hotmail.com	0846568789
surapong	surapong007@hotmail.com	0815678787
surapong	surapong007@hotmail.com	0834587890
wasinee	very_cute_girl@hotmail.com	0851231234
wasinee	very_cute_girl@hotmail.com	0878997890

Table ที่ถูกแก้ไขให้เป็น 1NF

Each attribute belongs to only one domain : แต่ละ Column (Attribute) ต้องอยู่ภายใน 1 Domain เท่านั้น

เรามาดูตัวอย่าง Table ที่ไม่ถูกกฎนี้กัน

USERNAME	COLUMN_NAME	VALUES
surapong	EMAIL	surapong007@hotmail.com
surapong	MOBILE_NO	0846568789
surapong	NICKNAME	BELL
wasinee	EMAIL	very_cute_girl@hotmail.com
wasinee	MOBILE_NO	0851231234
wasinee	NICKNAME	BELL

Column : VALUES มีค่ามาจากหลากหลาย Domain

จากตัวอย่าง Table จะเห็นว่า Column : VALUES นั้นมีค่ามาจากหลาย Domain คือจะเห็นว่า เป็นค่าที่มาจาก Domain : เบอร์มือถือ , Domain : email ปัญหาที่จะเกิดกับ Table นี้คือ ถ้าคุณ อยากระบุข้อมูล USERNAME , EMAIL , MOBILE_NO, NICKNAME ของ USER ทั้งหมดที่มี EMAIL = 'surapong007@hotmail.com' คุณจะเขียน SQL ยังไงออกมา (เขียนได้ นะครับแต่เขียนยากในระดับหนึ่งเลย)

อีกทั้ง Table นี้ไม่จัดเป็น Relation เพราะ ผลลัพธ์ของ Cartesian product of domains ที่ ออกมาใน attribute ต่างๆต้องมาจาก Domain เดียวกันไม่ใช่มาจากหลากหลาย Domain

วิธีแก้ไข

วิธีแก้ไขให้ Table นี้อยู่ในรูปแบบ 1NF คือแยก Column ที่มาจากหลาย Domain ให้เป็น Column ใหม่ไปเลยดังภาพด้านล่าง

USERNAME	EMAIL	MOBILE_NO	NICKNAME
surapong	surapong007@hotmail.com	0846568789	BELL
wasinee	very_cute_girl@hotmail.com	0851231234	BELL

Table ที่ถูกแก้ไขให้เป็น 1NF

จะเห็นว่า Table ถูกเพิ่ม Column : EMAIL , MOBILE_NO, NICKNAME เข้าไป

ลองตรวจสอบกับ TABLE : CURRENCY_TRADING_SYSTEM ว่าเป็น 1NF ไหม

TABLE : CURRENCY_TRADING_SYSTEM							
USERNAME (PK)	EMAIL	USER_TYPE	PRIORITY	CURRENCY_CODE (PK)	CURRENCY_NAME	COUNTRY	AMOUNT
surapong	surapong_inwza007@hotmail.com	BRONZE	100	THB	Baht	THAILAND	6000
sarun	fax_inwza007@hotmail.com	VIP	1000	THB	Baht	THAILAND	1000000
sarun	fax_inwza007@hotmail.com	VIP	1000	USD	US Dollar	UNITED STATES OF AMERICA	5000
sermsak	inwja@hotmail.com	BRONZE	100	JPY	Yen	JAPAN	100000
sermsak	inwja@hotmail.com	BRONZE	100	USD	US Dollar	UNITED STATES OF AMERICA	5000
apirat	ce_mocca@gmail.com	GOLD	500	THB	Baht	THAILAND	200000

TABLE : CURRENCY_TRADING_SYSTEM

เราลองตรวจ TABLE : CURRENCY_TRADING_SYSTEM ดูว่าเป็น 1NF ไหมโดยดูตามกฎ ของ 1NF ที่ว่า

- All of its attributes contain atomic values only ข้อนี้ผ่านเพราะทุกค่าของแต่ละ Column เป็น Atomic value
- Each attribute belongs to only one domain ข้อนี้ผ่านเพราะทุกค่าของแต่ละ Column มาจาก Domain เดียวกัน

ดังนั้น TABLE : CURRENCY_TRADING_SYSTEM จึงเป็น 1NF

2NF

A relation table structure T is in the second normal form (2NF) if and only if the relationship between primary key and every nonkey attribute is in full functional dependence

ผมแนะนำให้อ่านนิยามด้านบนครับเพราะเป็นนิยามที่ถูกต้อง แต่หากท่านไม่เข้าใจลองมาอ่านที่ผมแปลไทยดูครับ โดยคำแปลก็คือ

Table ที่จะเป็น 2NF ได้นั้น ความสัมพันธ์ระหว่าง Primary key กับ nonkey attribute ทุกตัวต้องเป็น Full functional dependence

ผมเชื่อว่าอ่านแล้วอาจจะไม่เข้าใจเพราะติดศัพท์ทางเทคนิค เรามาดูศัพท์เทคนิคทีละตัวครับ

Nonkey attribute

คือ Column หรือกลุ่มของ Column ที่ไม่ได้เป็น Candidate key (จริงๆ Column มันคือ Attribute นั้นแหละ ถ้าตามทฤษฎีเราต้องเรียก Column ว่า Attribute เรียก Row ว่า Tuple แต่ขอจะเป็น Column ละกันนะ)

Functional dependence (FD)

คือความสัมพันธ์แบบ Function อ่านแล้วงงใช่ไหมครับ คิดง่ายๆครับว่าถ้า A มีความสัมพันธ์แบบ Function กับ B (A -> B) แปลว่า ถ้าเรากำหนดค่า A เป็นค่าหนึ่ง จะได้ค่า B เป็นค่าหนึ่งเสมอ ซึ่งผมว่าก็ไม่เข้าใจอีกแหละ เรลองมาดูตัวอย่างอย่างงั้น

TABLE : CURRENCY_TRADING_SYSTEM

USERNAME (PK)	EMAIL	USER_TYPE	PRIORITY	CURRENCY_CODE (PK)	CURRENCY_NAME	COUNTRY	AMOUNT
surapong	surapong_inwza007@hotmail.com	BRONZE	100	THB	Baht	THAILAND	6000
sarun	fax_inwza007@hotmail.com	VIP	1000	THB	Baht	THAILAND	1000000
sarun	fax_inwza007@hotmail.com	VIP	1000	USD	US Dollar	UNITED STATES OF AMERICA	5000
sermsak	inwja@hotmail.com	BRONZE	100	JPY	Yen	JAPAN	100000
sermsak	inwja@hotmail.com	BRONZE	100	USD	US Dollar	UNITED STATES OF AMERICA	5000
asirat	ce_mocca@gmail.com	GOLD	500	THB	Baht	THAILAND	200000

ความสัมพันธ์ระหว่าง USERNAME กับ EMAIL

คุณลองดู COLUMN USERNAME กับ EMAIL ครับ คุณจะเห็นว่าถ้าเรากำหนดค่า USERNAME เป็น sarun เราจะได้ email เป็น fax_inwza007@hotmail.com เสมอ ซึ่งไม่ได้แค่ USERNAME เป็น sarun USERNAME อื่นๆก็เป็นแบบนี้ ความสัมพันธ์แบบนี้คือ Functional dependence ครับ ถ้ามว่าแล้วเราจะรู้ได้ใจว่ามันเป็น ต้องมานั่งไล่ดูค่าทุกค่าเลยหรือจริงๆคือไม่ใช่ครับ เราดูค่าเบื้องต้นเพื่อคาดเดาว่าอาจจะใช่ การจะบอกว่ใช่หรือไม่ใช่นั้นต้องไปดูความสัมพันธ์ตามงานจริงครับ คือในงานนี้ USERNAME กับ EMAIL เป็น Functional

dependence (USERNAME -> EMAIL) เพราะ USERNAME มี EMAIL ได้แค่ EMAIL เดียว ดังนั้นถ้ากำหนดค่า USERNAME อะไรก็จะอ้างอิงถึง EMAIL ได้ EMAIL เดียวเสมอ มันจะแตกต่างกับความสัมพันธ์ระหว่าง USERNAME กับ CURRENCY_CODE ครั้งที่ 1 USERNAME สามารถมีได้หลาย CURRENCY_CODE

TABLE : CURRENCY_TRADING_SYSTEM

USERNAME (PK)	EMAIL	USER_TYPE	PRIORITY	CURRENCY_CODE (PK)	CURRENCY_NAME	COUNTRY	AMOUNT
surapong	surapong_inwza007@hotmail.com	BRONZE	100	THB	Baht	THAILAND	6000
sarun	sax_inwza007@hotmail.com	VIP	1000	THB	Baht	THAILAND	1000000
sarun	sax_inwza007@hotmail.com	VIP	1000	USD	US Dollar	UNITED STATES OF AMERICA	5000
sermsak	mwja@hotmail.com	BRONZE	100	JPY	Yen	JAPAN	100000
sermsak	mwja@hotmail.com	BRONZE	100	USD	US Dollar	UNITED STATES OF AMERICA	5000
aspirat	ce_mocca@gmail.com	GOLD	500	THB	Baht	THAILAND	200000

ความสัมพันธ์ระหว่าง USERNAME กับ CURRENCY_CODE

Full Functional dependence

ในส่วนที่แล้วเรารู้ว่า Functional dependence (FD) แต่ Full Functional dependence คืออะไรละ อธิบายยากคุณลองดูตัวอย่างดีกว่า คุณสังเกตดูจะเห็นว่า { USERNAME , CURRENCY_CODE } มีความสัมพันธ์ FD กับ EMAIL

คุณลองดูได้เลยครับว่าจริงไหม มันจริงแน่นอนครับเพราะ USERNAME มันเป็น FD กับ EMAIL ดังนั้นจะเอา USERNAME ไปรวมกับ Column ไหนมันก็เป็น FD หหมดแหละ แต่สิ่งที่เราต้องการจริงๆคือความสัมพันธ์แบบ USERNAME -> EMAIL ไม่ใช่ { USERNAME , CURRENCY_CODE } -> EMAIL ดังนั้นจะเป็น Full Functional dependence ก็ต่อเมื่อ EMAIL นั้นขึ้นอยู่กับค่า USERNAME และ CURRENCY_CODE ทั้งคู่ ซึ่งจะเห็นว่าไม่ใช่เพราะ EMAIL ขึ้นอยู่กับค่า USERNAME ค่าเดียว

- { USERNAME , CURRENCY_CODE } -> EMAIL
- USERNAME -> EMAIL

หรือว่าง่ายๆถ้าหาค่าฝั่งซ้ายที่เป็น Subset ได้ก็แปลว่าไม่เป็น Full FD ละ จากตัวอย่าง { USERNAME , CURRENCY_CODE } -> EMAIL ไม่เป็น Full FD เพราะเราเจอ FD : USERNAME -> EMAIL ซึ่งจะเห็น USERNAME เป็น Subset ของ { USERNAME , CURRENCY_CODE }

ลองตรวจสอบกับ TABLE : CURRENCY_TRADING_SYSTEM ว่าเป็น 2NF หรือไม่

เรามา List แต่ละส่วนประกอบที่เราต้องใช้ตรวจสอบ

1. Primary key : { USERNAME , CURRENCY_CODE }

2. Nonkey attribute มีดังต่อไปนี้

- EMAIL
- USER_TYPE
- PRIORITY
- CURRENCY_NAME
- COUNTRY
- AMOUNT

3. Full Functional Dependency ของ Column (Attribute) ใน Table นี้

- USERNAME -> EMAIL
- USERNAME -> USER_TYPE
- USERNAME -> PRIORITY
- CURRENCY_CODE -> CURRENCY_NAME
- CURRENCY_CODE -> COUNTRY
- USER_TYPE -> PRIORITY
- { USERNAME , CURRENCY_CODE } -> AMOUNT

จากนิยาม

A relation table structure T is in the second normal form (2NF) if and only if the relationship between primary key and every nonkey attribute is in full functional dependence

Table ที่จะเป็น 2NF ได้นั้น ความสัมพันธ์ระหว่าง Primary key กับ nonkey attribute ทุกตัวต้องเป็น Full functional dependence

จะเห็นว่า TABLE : CURRENCY_TRADING_SYSTEM ไม่เป็น 2NF เพราะ ความสัมพันธ์ระหว่าง Primary key กับ nonkey attribute ไม่ได้เป็น Full functional dependence ดูได้จาก { USERNAME , CURRENCY_CODE } ไม่ได้เป็น Full FD กับ EMAIL , USER_TYPE , PRIORITY , CURRENCY_NAME , COUNTRY

วิธีแก้ไข

วิธีแก้ทำให้ Table นี้เป็น 2NF ก็คือแยก Table ออกไปโดยเราจะแยก Table ตาม FD ที่เป็น Subset ของ Primary เลยครับ จากนั้นก็ตรวจสอบอีกครั้งว่าเป็น 2NF ใหม่ โดยจากตัวอย่างเราจะแยกได้ 3 Table คือ

Table : USER

USERNAME (PK)	EMAIL	USER_TYPE	PRIORITY
surapong	surapong_inwza007@hotmail.com	BRONZE	100
sarun	fax_inwza007@hotmail.com	VIP	1000
sermsak	inwja@hotmail.com	BRONZE	100
apirat	ce_mocca@gmail.com	GOLD	500

TABLE : USER

ตรวจสอบว่า Table นี้เป็น 2NF ใหม่

1. Primary key : USERNAME

2. Nonkey attribute มีดังต่อไปนี้

- USER_TYPE
- PRIORITY
- คุณอาจจะสงสัยว่าทำไม EMAIL หายไปไหน ที่มันหายไปเพราะ EMAIL เป็น Candidate key นะครับ

3. Full Functional Dependency ของ Column (Attribute) ใน Table นี้

- USERNAME -> USER_TYPE
- USERNAME -> PRIORITY
- USER_TYPE -> PRIORITY

Table นี้เป็น 2NF เพราะ ความสัมพันธ์ระหว่าง Primary key กับ nonkey attribute เป็น Full FD ทั้งหมด

Table : CURRENCY

CURRENCY_CODE (PK)	CURRENCY_NAME	COUNTRY
THB	Baht	THAILAND
USD	US Dollar	UNITED STATES OF AMERICA
JPY	Yen	JAPAN

Table : CURRENCY

ตรวจสอบว่า Table นี้เป็น 2NF ใหม่

1. Primary key : CURRENCY_CODE

2. Nonkey attribute มีดังต่อไปนี้

- COUNTRY
- คุณอาจจะสงสัยว่าทำไม CURRENCY_NAME หายไปไหน ที่มันหายไปเพราะ CURRENCY_NAME เป็น Candidate key นะครับ ทำไม Country ไม่เป็น ที่ไม่เป็นก็เพราะบางประเทศมีหลายสกุลเงินนะครับ

3. Full Functional Dependency ของ Column (Attribute) ใน Table นี้

- CURRENCY_CODE -> COUNTRY

Table : USER_CURRENCY

USERNAME (PK)	CURRENCY_CODE (PK)	AMOUNT
surapong	THB	6000
sarun	THB	1000000
sarun	USD	5000
sermsak	JPY	100000
sermsak	USD	5000
apirat	THB	200000

Table : USER_CURRENCY

ตรวจสอบว่า Table นี้เป็น 2NF ใหม่

1. Primary key : { USERNAME , CURRENCY_CODE }

2. Nonkey attribute มีดังต่อไปนี้

- COUNTRY

- คุณอาจจะสงสัยว่าทำไม CURRENCY_NAME หายไปไหน ที่มันหายไปเพราะ CURRENCY_NAME เป็น Candidate key นะครับ ทำไม Country ไม่เป็นละ ที่ไม่เป็นก็เพราะบางประเทศมีหลายสกุลเงินครับ

3. Full Functional Dependency ของ Column (Attribute) ใน Table นี้

- { USERNAME , CURRENCY_CODE } -> AMOUNT

Table นี้ 2NF เพราะ ความสัมพันธ์ระหว่าง Primary key กับ nonkey attribute เป็น Full FD ทั้งหมด

3NF

A relation table structure T is in the third normal form (3NF) if and only if T is in 2NF and there are no functional dependencies between nonkey attributes of T

ผมแนะนำให้อ่านนิยามด้านบนครับเพราะเป็นนิยามที่ถูกต้อง แต่ถ้าไม่เข้าใจลองอ่านที่ผมพยายามแปลเป็นไทย

Table จะเป็น 3NF ก็ต่อเมื่อ Table นั้นเป็น 2NF และ Table นั้นไม่มี functional dependence ระหว่าง nonkey attributes ของ Table นั้น

สำหรับขั้นนี้นั้นเราไม่ติดศัพท์อะไรแล้ว เรามาตรวจสอบ Table ที่เราได้จากขั้นตอนที่แล้วกัน เลยว่าจะสามารถเป็น 3NF ได้หรือไม่

Table : USER_CURRENCY

USERNAME (PK)	CURRENCY_CODE (PK)	AMOUNT
surapong	THB	6000
sarun	THB	1000000
sarun	USD	5000
sermsak	JPY	100000
sermsak	USD	5000
apirat	THB	200000

Table : USER_CURRENCY

ตรวจสอบว่า Table นี้เป็น 3NF ไหม

1. Nonkey attribute มีดังต่อไปนี้

- COUNTRY

2. FD ระหว่าง Nonkey attribute

ไม่มีเพราะมี Nonkey attribute ตัวเดียว

เนื่องจาก Table นี้ไม่มี functional dependence ระหว่าง nonkey attributes และ Table นี้ก็เป็น 2NF ดังนั้น Table จึงเป็น 3NF

Table : CURRENCY

CURRENCY_CODE (PK)	CURRENCY_NAME	COUNTRY
THB	Baht	THAILAND
USD	US Dollar	UNITED STATES OF AMERICA
JPY	Yen	JAPAN

Table : CURRENCY

ตรวจสอบว่า Table นี้เป็น 3NF ใหม่

1. Nonkey attribute มีดังต่อไปนี้

- COUNTRY

2. FD ระหว่าง Nonkey attribute

ไม่มีเพราะมี Nonkey attribute ตัวเดียว

เนื่องจาก Table นี้ไม่มี functional dependence ระหว่าง nonkey attributes และ Table นี้ก็เป็น 2NF ดังนั้น Table จึงเป็น 3NF

Table : USER

USERNAME (PK)	EMAIL	USER_TYPE	PRIORITY
surapong	surapong_inwza007@hotmail.com	BRONZE	100
sarun	fax_inwza007@hotmail.com	VIP	1000
sermsak	inwja@hotmail.com	BRONZE	100
apirat	ce_mocca@gmail.com	GOLD	500

Table : USER

ตรวจสอบว่า Table นี้เป็น 3NF ไหม

1. Nonkey attribute มีดังต่อไปนี้

- USER_TYPE
- PRIORITY

2. FD ระหว่าง Nonkey attribute

- USER_TYPE -> PRIORITY

Table นี้ไม่เป็น 3NF เพราะเราตรวจพบ FD คือ USER_TYPE -> PRIORITY ซึ่งเป็น Nonkey attribute

วิธีแก้ไข

ให้ทำการแยกตารางออกไปตาม FD ของ Nonkey attribute ครั้น ส่วน Table เดิมเก็บ Column ที่เป็นตัวกำหนดไว้ ส่วน Column ที่มีค่าตามให้ลบออกจาก Table โดยจากตัวอย่างเราจะแก้ไข Table : USER ลบ Column : PRIORITY และ สร้าง Table USER_TYPE ขึ้นมาใหม่

Table : USER

USERNAME (PK)	EMAIL	USER_TYPE
surapong	surapong_inwza007@hotmail.com	BRONZE
sarun	fax_inwza007@hotmail.com	VIP
sermsak	inwja@hotmail.com	BRONZE
apirat	ce_mocca@gmail.com	GOLD

Table : USER

ตรวจสอบว่า Table นี้เป็น 3NF ไหม

1. Nonkey attribute มีดังต่อไปนี้

- USER_TYPE

2. FD ระหว่าง Nonkey attribute

ไม่มีเพราะมี Nonkey attribute ตัวเดียว

เนื่องจาก Table นี้ไม่มี functional dependence ระหว่าง nonkey attributes และ Table นี้ก็เป็น 2NF ดังนั้น Table จึงเป็น 3NF ตามนิยาม

Table : USER_TYPE

USER_TYPE (PK)	PRIORITY
BRONZE	100
VIP	1000
GOLD	500

Table : USER_TYPE

ตรวจสอบว่า Table นี้เป็น 3NF ไหม

1. Nonkey attribute มีดังต่อไปนี้

- PRIORITY

2.FD ระหว่าง Nonkey attribute

ไม่มีเพราะมี Nonkey attribute ตัวเดียว

เนื่องจาก Table นี้ไม่มี functional dependence ระหว่าง nonkey attributes และ Table นี้ก็เป็น 2NF ดังนั้น Table จึงเป็น 3NF ตามนิยาม

Table ที่เป็น 3NF ที่มีปัญหา INSERT , DELETE , UPDATE

USERNAME (PK)	EMAIL	CURRENCY_CODE (PK)	AMOUNT
surapong	surapong_inwza007@hotmail.com	THB	6000
sarun	fax_inwza007@hotmail.com	THB	1000000
sarun	fax_inwza007@hotmail.com	USD	5000
sermsak	inwja@hotmail.com	JPY	100000
sermsak	inwja@hotmail.com	USD	5000
apirat	ce_mocca@gmail.com	THB	200000

Table ที่เป็น 3NF และมีปัญหา INSERT , DELETE , UPDATE

เรามาดูรายละเอียดของ TABLE นี้เพื่อนำไปใช้ในการตรวจสอบว่า Table นี้เป็น 3NF ไหม

1. Primary key : { USERNAME , CURRENCY_CODE }

2. Candidate key : { USERNAME , CURRENCY_CODE } , { EMAIL, CURRENCY_CODE }

3. Nonkey attribute มีดังต่อไปนี้

- AMOUNT EMAIL ไม่เป็น Nonkey attribute เพราะมันเป็น 1 ใน Candidate key

4.FD ใน Table

- { USERNAME , CURRENCY_CODE } -> AMOUNT
- { EMAIL , CURRENCY_CODE } -> AMOUNT
- USERNAME -> EMAIL
- EMAIL -> USERNAME

ตรวจสอบ NF แต่ละชั้น

- 1NF เป็น 1NF เพราะทุก Column (Attribute) เป็น Atomic และ แต่ละ Column มีค่ามาจาก Domain เดียว
- 2NF
เป็น 2NF เพราะ ความสัมพันธ์ระหว่าง Primary key กับ nonkey attribute ทุกตัวเป็น Full functional dependence ดูได้จาก { USERNAME , CURRENCY_CODE } เป็น Full FD กับ AMOUNT ซึ่งเป็น nonkey attribute ตัวเดียว
- 3NF เป็น 3NF เพราะ Table นี้เป็น 2NF และ Table นี้ไม่มี functional dependence ระหว่าง nonkey attributes ดูได้จาก Nonkey attribute มีตัวเดียว

ปัญหา Insert ข้อมูลไม่ถูกต้องเข้าไปได้

USERNAME (PK)	EMAIL	CURRENCY_CODE (PK)	AMOUNT
surapong	surapong_inwza007@hotmail.com	THB	6000
sarun	fax_inwza007@hotmail.com	THB	1000000
sarun	fax_inwza007@hotmail.com	USD	5000
sermsak	inwja@hotmail.com	JPY	100000
sermsak	inwja@hotmail.com	USD	5000
apirat	ce_mocca@gmail.com	THB	200000
apirat	champ007@gmail.com	JPY	1000000

ตัวอย่างปัญหา Insert ข้อมูลไม่ถูกต้องเข้าไปได้

Table นี้จะมีปัญหาการ Insert ข้อมูลที่ไม่ถูกต้องเข้าไปได้ที่เราสามารถ Insert ข้อมูล EMAIL ของ USERNAME : apirat เป็น champ007@gmail.com ได้ ซึ่งนั่นทำให้เกิดปัญหา Data inconsistency

ปัญหา Insert ข้อมูลเข้าไปไม่ได้

จากตัวอย่างเราไม่สามารถ Insert ข้อมูลของ User ใหม่เข้าไปได้เลย เช่น ถ้าผมต้องการ Insert ข้อมูล User : wasinee เข้าไปผมจะไม่สามารถเข้าไปได้เพราะติดกฎ Primary key must not be null

USERNAME (PK)	EMAIL	CURRENCY_CODE (PK)	AMOUNT
wasinee	very_cute_girl@hotmail.com	NULL	NULL

ตัวอย่างปัญหา Insert ข้อมูลเข้าไปไม่ได้

ปัญหา Delete ข้อมูล

จากตัวอย่างถ้าเราลบข้อมูลการ Row ที่ผมล้อมกรอบสีแดงไว้ ข้อมูลของ User : apirat จะหายไปเลย ทั้งๆที่เราอยากลบข้อมูลการที่เขามีเงินสกุล THB ไปเท่านั้น

USERNAME (PK)	EMAIL	CURRENCY_CODE (PK)	AMOUNT
surapong	surapong_inwza007@hotmail.com	THB	6000
sarun	fax_inwza007@hotmail.com	THB	1000000
sarun	fax_inwza007@hotmail.com	USD	5000
sermsak	inwja@hotmail.com	JPY	100000
sermsak	inwja@hotmail.com	USD	5000
apirat	ce_mocca@gmail.com	THB	200000

ตัวอย่างปัญหา Delete ข้อมูล

ซึ่งจากปัญหานี้จึงต้องทำ Normalization ขั้นตอนต่อไป

BCNF

A relation table structure T is in the Boyce/Codd normal form (BCNF) if and only determinant in T is a candidate key of T.

ผมแนะนำให้อ่านนิยามด้านบนครับเพราะเป็นนิยามที่ถูกต้อง แต่หากท่านไม่เข้าใจลองมาอ่านที่ผมแปลไทยดูครับ โดยคำแปลก็คือ

Table จะเป็น BCNF ได้นั้น Determinant ทั้งหมดใน Table จะต้องเป็น Candidate key ของ Table